

BC01 — Socle de données anti-fraude

architecture et fonctionnement

Comment l'entrepôt en étoile, le pipeline d'ingestion batch/streaming et les garanties de qualité s'articulent pour produire une source de vérité unique, fiable et rejouable.

Bloc de compétences : BC01 — Architecture Data Warehouse

Périmètre : PostgreSQL (staging/dwh/mart/ops), Redis, Apache Kafka

Données : 5 000 clients simulés · 400 000 transactions · 6 000 fraudes (1,50 %)

Document : note d'architecture et guide de fonctionnement

Sommaire

1. Objectif du bloc BC01
2. Argumentaire d'architecture
3. Le schéma en étoile
4. Fonctionnement étape par étape
5. Garanties techniques
6. Contrôles qualité
7. Limites connues et perspectives

1. Objectif du bloc BC01

Le bloc BC01 constitue la fondation data de l'ensemble du projet de détection de fraude : c'est lui qui décide où et comment vit la donnée avant que les blocs suivants (analyse exploratoire, modèles supervisés, deep learning, API temps réel) ne viennent la consommer. Concrètement, il répond à trois questions posées par le cahier des charges : comment modéliser les données d'une banque de détail sous une forme exploitable analytiquement (architecture en étoile), comment simuler l'arrivée continue de transactions comme le ferait un système de paiement réel (ingestion streaming via Kafka), et où stocker ces données pour qu'elles soient à la fois interrogeables en masse (PostgreSQL) et accessibles en quelques millisecondes pour un scoring en production (Redis).

Le choix a été fait de travailler sur des données **entièrement synthétiques**, générées par un script dédié plutôt que sur un jeu de données Kaggle existant. Les jeux de données publics de détection de fraude les plus connus (par exemple le jeu ULB « Credit Card Fraud Detection ») anonymisent les variables par une analyse en composantes principales : il n'y reste qu'un identifiant temporel, un montant et 28 colonnes numériques sans signification métier. Un tel jeu de données interdit de facto la construction d'un schéma en étoile porteur de sens (aucune notion de client, de marchand ou de canal de paiement) et bloque le profiling comportemental prévu au BC02. Générer les données permet au contraire de maîtriser de bout en bout le schéma relationnel, les typologies de fraude injectées et la volumétrie, tout en conservant une architecture techniquement identique à celle qu'exigerait un jeu de données réel.

2. Argumentaire d'architecture

L'architecture retenue s'organise en quatre couches inspirées de la méthode Kimball — `staging`, `dwh`, `mart` et `ops` — plutôt qu'en un schéma unique et plat. Ce choix n'est pas un exercice de style académique : il répond à une contrainte concrète du projet, celle de devoir alimenter la même table de faits par deux voies radicalement différentes (une génération de masse et un flux Kafka message par message) sans que ces deux voies puissent un jour produire des résultats divergents. La couche `staging` sert de zone d'atterrissage commune : qu'une transaction provienne du script de génération batch ou d'un message Kafka, elle est d'abord écrite telle quelle, sans aucune règle métier, dans la table `staging.raw_transactions`. C'est seulement ensuite qu'une fonction SQL unique, `dwh.charger_faits_transactions()`, vient résoudre les dimensions et écrire dans la couche `dwh`. Parce que batch et streaming appellent littéralement la même fonction, il devient structurellement impossible que la logique de transformation diverge entre les deux voies d'ingestion — un risque pourtant fréquent dans les architectures qui dupliquent le code de transformation entre un job batch et un consommateur temps réel.

La couche `mart` assume un rôle volontairement restreint : elle n'expose que des vues SQL en lecture, jamais de tables physiques, de sorte que les blocs consommateurs (BC02 à BC06) disposent d'un contrat d'interface stable même si le modèle interne du `dwh` évolue. La couche `ops`, enfin, est

transversale par construction : elle journalise chaque chargement (`ops.log_chargement`) et le résultat de chaque contrôle qualité (`ops.controle_qualite`) sans jamais se trouver sur le chemin critique d'écriture — un incident sur le journal ne peut pas faire échouer un chargement de données, ce qui aurait été le cas si l'audit avait été implémenté par un trigger synchrone sur la table de faits.

Deux décisions de modélisation méritent d'être justifiées séparément. La première concerne le traitement des dimensions qui ne trouvent pas de correspondance : plutôt que de laisser une transaction sans fait lorsque son horodatage ou son canal ne matche aucune ligne de dimension, le schéma réserve une clé technique `0` — un membre « Inconnu » — sur `dim_temps` et `dim_canal`. La fonction de transformation s'y replie automatiquement (`LEFT JOIN` suivi d'un `COALESCE`) : la transaction est toujours écrite, avec une dégradation visible et traçable plutôt qu'une perte silencieuse. Ce traitement n'est en revanche pas appliqué à la relation client, volontairement laissée en jointure stricte, parce qu'un identifiant client absent du référentiel est un vrai défaut d'intégrité — pas un cas limite acceptable — qui doit remonter comme anomalie plutôt qu'être masqué. La seconde décision concerne l'agence bancaire : elle est modélisée comme un attribut du client (`dim_clients.agence_key`) et non comme une dimension de la transaction, parce qu'une transaction carte ne transite pas par une agence physique — c'est le compte, pas l'achat, qui est rattaché à une agence gestionnaire.

Le choix des deux moteurs de stockage répond à deux besoins distincts et complémentaires. PostgreSQL porte la vérité historique et interrogeable : l'ensemble du schéma en étoile, avec ses contraintes d'intégrité et ses possibilités d'agrégation SQL, en fait l'outil naturel pour l'analyse exploratoire du BCo2 et l'entraînement des modèles du BCo3/BCo4. Redis, à l'inverse, ne conserve qu'un état éphémère et pré-agrégé — le nombre de transactions et le montant cumulé sur la dernière heure par client — mis à jour à chaque événement streaming et expirant automatiquement au bout d'une heure. Cette redondance délibérée est ce qui permettra, au BCo5, à l'API de scoring de lire une feature comportementale en une opération mémoire au lieu d'une agrégation SQL, pour tenir la contrainte de décision en moins de 200 millisecondes. Kafka, enfin, tourne en mode KRaft (sans Zookeeper) : ce choix simplifie l'infrastructure d'un cluster à un seul broker, suffisant pour démontrer le mécanisme d'ingestion streaming sans complexité opérationnelle superflue à ce stade du projet.

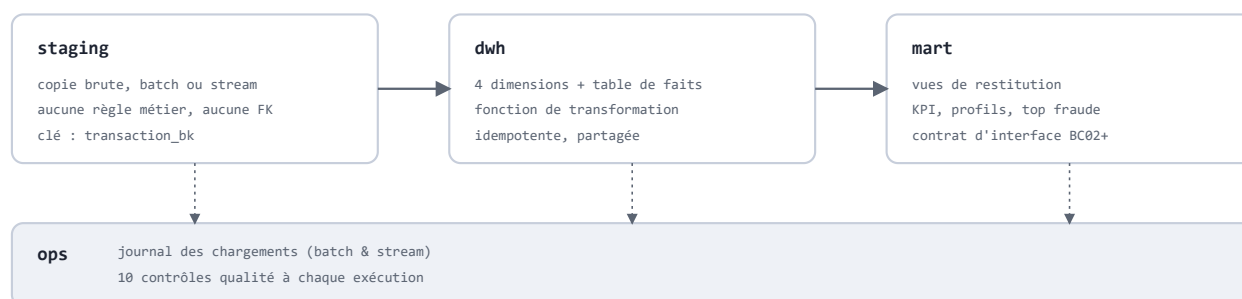


Figure 1 — Les quatre couches du socle. La couche ops observe les trois autres sans jamais se trouver sur le chemin critique d'écriture.

3. Le schéma en étoile

La table de faits `dwh.fact_transactions` porte le grain le plus fin du modèle : une ligne par transaction carte. Elle référence trois dimensions par clé étrangère — `dim_clients`, `dim_temps` et `dim_canal` — et porte deux attributs dégénérés essentiels à la suite du projet : `is_fraud` et `fraud_type`, tous deux *nullables*. Cette nullabilité n'est pas un oubli : une transaction issue du flux streaming entre dans la table de faits dès que son horodatage est résolu, alors même que son statut de fraude n'est pas encore connu — dans la réalité, une fraude n'est confirmée qu'après investigation ou réclamation du client, parfois des jours plus tard. Les transactions batch, à l'inverse, simulent un historique déjà clos et arrivent donc déjà labellisées.

TABLE	RÔLE
<code>dim_clients</code>	5 000 clients pseudonymisés (aucun nom généré) — segment, revenu, agence gestionnaire
<code>dim_agences</code>	10 agences du réseau ; rattachée à <code>dim_clients</code> , jamais directement au fait
<code>dim_temps</code>	1 ligne par heure sur l'année simulée, + le membre « Inconnu » (clé 0)
<code>dim_canal</code>	combinaison canal de paiement / code MCC, + le membre « Inconnu » (clé 0)
<code>fact_transactions</code>	grain = 1 transaction ; montant, distance au domicile, authentification, statut de fraude nullable
<code>fact_card_blocks</code>	historique des blocages carte — feature comportementale exploitée au BC02

4. Fonctionnement étape par étape

Le pipeline s'exécute dans un ordre précis, chaque étape dépendant de la précédente. Voici la séquence complète, de l'infrastructure vide jusqu'à un socle interrogeable et vérifié.

1 Démarrage de l'infrastructure

`docker compose up -d` démarre cinq conteneurs : PostgreSQL (port 5433), Redis (6379), Kafka en mode KRaft (9092), Adminer pour l'inspection visuelle de la base (8080) et Kafka UI pour celle des topics (8090). Au tout premier démarrage, PostgreSQL exécute automatiquement, dans l'ordre alphabétique, les six scripts du dossier `bc01_socle_donnees/sql/` : création des quatre schémas, du schéma en étoile, de la table de staging, des tables d'audit `ops`, de la fonction de transformation, puis des vues `mart`.

2 Génération et chargement batch

`data/seed.py` vide les tables (`TRUNCATE ... RESTART IDENTITY`), recrée les membres « Inconnu », puis charge dans l'ordre les référentiels stables (10 agences, 9 combinaisons canal/MCC, 8 760 lignes horaires pour l'année 2025, 5 000 clients). Il génère ensuite 400 000 transactions de façon vectorisée avec `numpy`, en injectant six typologies de fraude aux signatures statistiquement distinctes, et les écrit par lot (`COPY`) dans `staging.raw_transactions` avec une clé métier déterministe (`TX-000000001`, ...). Le script appelle enfin `dwh.charger_faits_transactions('batch')`, qui résout les dimensions et peuple la table de faits, et journalise l'ensemble de l'opération dans `ops.log_chargement`.

3 Publication du flux streaming

`streaming/producer.py` pioche des identifiants clients réels dans `dwh.dim_clients` et publie sur le topic Kafka `transactions.raw` un événement JSON par transaction simulée, à intervalle aléatoire pour imiter un flux réel. La clé métier de chaque transaction est générée une seule fois, au moment de la production, et voyage avec le message : une re-livraison du même message par Kafka portera toujours la même clé, condition nécessaire à la déduplication en aval.

4 Ingestion du flux par micro-lots

`streaming/consumer.py` lit le topic en continu et traite les messages par lots de 200 (ou toutes les deux secondes). Chaque message est d'abord validé contre un schéma JSON minimal ; un message invalide part immédiatement vers le topic `transactions.dlq` (file de rejets) au lieu de bloquer le flux. Chaque message valide est ensuite vérifié contre un jeu Redis à expiration pour écarter les doublons déjà traités. Le lot restant est écrit dans `staging.raw_transactions` (`ON CONFLICT DO NOTHING`), puis la même fonction `dwh.charger_faits_transactions('stream')` qu'à l'étape 2 est invoquée. Ce n'est qu'une fois cette écriture réussie que le consumer met à jour le feature store Redis (compteurs de transactions et de montant sur la dernière heure) et committe les offsets Kafka correspondants — jamais avant, pour garantir qu'un échec transitoire ne fasse perdre aucun message.

5 Contrôle qualité du socle

`scripts/quality_checks.py` exécute dix vérifications SQL sur l'état courant du socle — complétude des clés étrangères, intégrité référentielle, unicité des clés métier, cohérence des montants et des labels, plausibilité du taux de fraude, fraîcheur du staging, conformité des référentiels — et journalise chaque résultat dans `ops.controle_qualite`. Ce script est pensé pour être rejoué après chaque chargement, batch comme streaming, afin de détecter une régression avant qu'elle n'atteigne les blocs suivants.

6 Mise à disposition en aval

Les vues `mart.v_kpis_globaux`, `mart.v_kpis_ingestion`, `mart.v_profil_horaire` et `mart.v_top_fraude_type` exposent le socle aux blocs suivants sans qu'ils aient à connaître le détail du modèle dimensionnel sous-jacent — c'est ce contrat d'interface que le BCo2 interrogera pour ses statistiques descriptives et son profiling des fraudeurs.

400 000

transactions ingérées
(batch)

6 000

fraudes — 1,50 %

10 / 10

contrôles qualité au vert

6

typologies de fraude
simulées

5. Garanties techniques

La fiabilité du pipeline ne repose pas sur une hypothèse de fonctionnement sans incident, mais sur des mécanismes explicites qui rendent chaque rejeu sans danger. Rejouer `data/seed.py` ne double jamais les données grâce au `TRUNCATE` initial combiné à une clé métier déterministe ; rejouer un lot Kafka déjà traité n'insère aucun doublon grâce à la contrainte `ON CONFLICT (transaction_bk) DO NOTHING`, doublée d'un jeu de déduplication Redis. Les offsets Kafka ne sont committés qu'après l'écriture réussie en base (sémantique *at-least-once*), et le marquage Redis n'intervient lui aussi qu'après ce succès : un lot en échec est donc rejoué à l'identique au prochain cycle, sans qu'aucun message ne soit perdu ni faussement considéré comme déjà traité. Enfin, le producer active `enable.idempotence` côté Kafka, qui élimine les doublons introduits par les retentatives réseau du client lui-même.

PROPRIÉTÉ	MÉCANISME
Idempotence batch	<code>TRUNCATE ... RESTART IDENTITY</code> + clé métier déterministe
Idempotence stream	<code>ON CONFLICT DO NOTHING</code> + déduplication Redis (TTL 24 h)
At-least-once	commit de l'offset Kafka après écriture réussie en base, jamais avant
Pas de perte sur échec transitoire	marquage Redis différé après succès Postgres — un lot en échec est rejoué à l'identique
Messages malformés isolés	validation de schéma JSON — rejet vers <code>transactions.dlq</code>
Pas de perte silencieuse de dimension	membre « Inconnu » sur <code>dim_temps</code> / <code>dim_canal</code>

6. Contrôles qualité

Les dix contrôles exécutés par `quality_checks.py` couvrent cinq catégories : complétude (aucune clé étrangère nulle, aucune ligne de staging bloquée plus de dix minutes, conformité numérique des référentiels), intégrité (tout `client_id` du staging résout vers `dim_clients`), unicité (aucun doublon de clé métier dans la table de faits), cohérence (montants et distances non négatifs, label de fraude cohérent avec le type de fraude) et plausibilité (le taux de fraude observé reste dans la fourchette attendue de 0,5 % à 3 %). Sur le dernier chargement complet, les dix contrôles sont au vert.

7. Limites connues et perspectives

Trois limites sont assumées à ce stade du projet. D'abord, le membre « Inconnu » de `dim_temps` sera systématiquement mobilisé par toute démonstration du flux streaming lancée en dehors de l'année simulée (2025) : le comportement est correct — aucune transaction n'est perdue — mais il masque la fraîcheur temporelle réelle de la démonstration ; une version de production régénérerait cette dimension sur une fenêtre glissante. Ensuite, Kafka tourne en cluster à un seul broker, suffisant pour démontrer le mécanisme d'ingestion mais pas pour un passage à l'échelle. Enfin, la fenêtre Redis des features en ligne (`INCR` / `EXPIRE`) est une fenêtre glissante approximative : une implémentation exacte à la seconde utiliserait un ensemble trié horodaté (`ZADD` / `ZREMRANGEBYSCORE`). Ces points sont documentés plutôt que corrigés maintenant parce qu'ils n'affectent pas la validité des blocs suivants et représentent des axes d'amélioration explicites pour une mise en production réelle.